```
import numpy as np

# Parameter Harmony Search
HMS = 10 # ukuran harmony memory
HMCR = 0.9 # harmony memory considering rate
PAR = 0.3 # pitch adjustment rate
NI = 100 # jumlah iterasi

# Parameter Jaringan/Masalah
area_size = 100
num_router = 3
num_node = 20

# Inisialisasi posisi node jaringan
nodes = np.random.rand(num_node, 2) * area_size
```

```
# Fungsi fitness (delay + cost + coverage)
def fitness(routers):
    # Menghitung rata-rata delay (jarak terpendek dari setiap node ke router terdekat)
    delay = np.mean([np.min(np.linalg.norm(nodes - r, axis=1)) for r in routers])

    # Menghitung coverage (persentase node yang dicakup oleh setidaknya satu router)
    # Asumsi radius cakupan adalah 20 (jarak < 20)
    coverage = np.mean([np.any(np.linalg.norm(nodes - r, axis=1) < 20) for r in routers])

    # Menggabungkan delay dan coverage (minimasi)
    # 0.7*delay + 0.3*(1 - coverage)
    # (1 - coverage) digunakan karena kita ingin memaksimalkan coverage
    return 0.7*delay + 0.3*(1 - coverage)
```

```
# Inisialisasi Harmony Memory (HM)
# HM berisi HMS harmoni (set posisi router)
HM = [np.random.rand(num_router, 2) * area_size for _ in range(HMS)]
HM_fitness = [fitness(h) for h in HM]

# Loop Utama Harmony Search
for it in range(NI):
    new_harmony = []

    # Membangun Harmoni Baru
    for r in range(num_router):
        if np.random.rand() < HMCR:
            # Memilih dari Harmony Memory (Memori)
            idx = np.random.randint(HMS)
            pos = HM[idx][r]

            # Pitch Adjustment (Penyesuaian Nada)
            if np.random.rand() < PAR:
                pos += np.random.uniform(-1, 1, 2)
        else:
            # Memilih secara acak (Random)
            pos = np.random.rand(2) * area_size
        new_harmony.append(pos)

    new_harmony = np.array(new_harmony)
    new_fit = fitness(new_harmony)

    # Update Harmony Memory
    # Mencari harmoni terburuk di HM
    worst_idx = np.argmax(HM_fitness)

    # Jika harmoni baru lebih baik (fitness lebih kecil) dari yang terburuk
    if new_fit < HM_fitness[worst_idx]:
        HM[worst_idx] = new_harmony
        HM_fitness[worst_idx] = new_fit
```

```
# Hasil Akhir
# Mengambil harmoni (posisi router) dengan fitness terbaik
best = HM[np.argmin(HM_fitness)]

print("Best router positions:\n", best)
print("Best fitness:", np.min(HM_fitness))
```

```
Best router positions:
 [[54.04073784 79.79582157]
 [31.97231264 88.70031848]
 [48.93913695 69.74598285]]
Best fitness: 1.5639163016036304
```

```
import numpy as np

# Parameter Harmony Search
HMS = 10 # ukuran harmony memory
HMCR = 0.9 # harmony memory considering rate
PAR = 0.3 # pitch adjustment rate
NI = 100 # jumlah iterasi

# Parameter Jaringan/Masalah
area_size = 100
num_router = 3
num_node = 20

# Inisialisasi posisi node jaringan
nodes = np.random.rand(num_node, 2) * area_size

# Fungsi fitness (delay + cost + coverage)
def fitness(routers):
    # Menghitung rata-rata delay (jarak terpendek dari setiap node ke router terdekat)
    delay = np.mean([np.min(np.linalg.norm(nodes - r, axis=1)) for r in routers])

    # Menghitung coverage (persentase node yang dicakup oleh setidaknya satu router)
    # Asumsi radius cakupan adalah 20 (jarak < 20)
    coverage = np.mean([np.any(np.linalg.norm(nodes - r, axis=1) < 20) for r in routers])

    # Menggabungkan delay dan coverage (minimasi)
    # 0.7*delay + 0.3*(1 - coverage)
    # (1 - coverage) digunakan karena kita ingin memaksimalkan coverage
    return 0.7*delay + 0.3*(1 - coverage)

# Inisialisasi Harmony Memory (HM)
# HM berisi HMS harmoni (set posisi router)
HM = [np.random.rand(num_router, 2) * area_size for _ in range(HMS)]
HM_fitness = [fitness(h) for h in HM]

# Loop Utama Harmony Search
for it in range(NI):
    new_harmony = []

    # Membangun Harmoni Baru
    for r in range(num_router):
        if np.random.rand() < HMCR:
            # Memilih dari Harmony Memory (Memori)
            idx = np.random.randint(HMS)
            pos = HM[idx][r]

            # Pitch Adjustment (Penyesuaian Nada)
            if np.random.rand() < PAR:
                pos += np.random.uniform(-1, 1, 2)
        else:
            # Memilih secara acak (Random)
            pos = np.random.rand(2) * area_size
        new_harmony.append(pos)

    new_harmony = np.array(new_harmony)
    new_fit = fitness(new_harmony)

    # Update Harmony Memory
    # Mencari harmoni terburuk di HM
    worst_idx = np.argmax(HM_fitness)

    # Jika harmoni baru lebih baik (fitness lebih kecil) dari yang terburuk
    if new_fit < HM_fitness[worst_idx]:
        HM[worst_idx] = new_harmony
        HM_fitness[worst_idx] = new_fit

# Hasil Akhir
# Mengambil harmoni (posisi router) dengan fitness terbaik
best = HM[np.argmin(HM_fitness)]

print("Best router positions:\n", best)
print("Best fitness:", np.min(HM_fitness))
```

```
Best router positions:
 [[91.96468864 21.96030426]
 [43.16439673 69.25706734]
 [91.89584839  4.8940238 ]]
Best fitness: 1.407652941588577
```